

Leveraging Critical Proof Obligations for Efficient IC3 Verification

Lingfeng Zhu^{1,2}, Xindi Zhang^{1,2}, Yongjian Li^{1,2,*}, Shaowei Cai^{1,2}

¹Key Laboratory of System Software (Chinese Academy of Sciences) and State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences

²School of Computer Science and Technology, University of Chinese Academy of Sciences
{zhulf, zhangxd, lyj238, caisw}@ios.ac.cn

Abstract—IC3 and its variants are SAT-based model-checking methods that play a critical role in hardware verification. Efficient management of proof obligations, which track states that need to be proven unreachable, is essential for improving verification performance. This paper presents a novel approach that utilizes Critical Proof Obligations (CPOs) to improve proof obligation management. We propose two techniques, CPO-Driven UNSAT Core Generation and CPO-Driven Proof Obligation Propagation, to promote lemma propagation and frame refinement. Experimental results on HWMCC benchmarks demonstrate significant improvements in CPO discovery and lemma propagation, resulting in notable performance gains.

Index Terms—model checking, IC3, critical proof obligation, assumption core consistency

I. INTRODUCTION

Safety model checking (MC) automatically determines whether a transition system $S = \langle V, I, T \rangle$ satisfies a specified safety property P . MC is a fundamental software and hardware verification challenge with extensive theoretical and practical applications [9]. IC3 [8] (also known as PDR [10]) is a prominent symbolic MC algorithm that leverages high-performance incremental SAT solvers and sophisticated induction-based interpolation sequence refinement techniques. In recent years, IC3 and its variants [11, 15, 18] have gained significant attention due to their completeness and scalability compared to Bounded Model Checking (BMC) and Interpolation-based Model Checking (IMC).

IC3 maintains an over-approximation sequence known as *frames*, incrementally constructing and refining them to either generate a proof for the specified safety property or provide a counterexample. This refinement is achieved by recursively blocking bad cubes, which are counterexamples to induction (CTIs) capable of reaching the negation of the safety property. During this process, proof obligations (POs) are generated to track states that need to be proven unreachable. These POs are then generalized to produce lemmas, which are added to the frames and propagated to subsequent frames, reducing redundancy and accelerating convergence.

Lemma propagation plays a crucial role in the efficiency of IC3, as highlighted by several studies. [13] explore blocking states farther from the bad states to discover stronger inductive lemmas, enabling deeper generalization and enhancing lemma

propagation in IC3. [22] directs induction generalization towards propagating *i*-good lemmas to accelerate convergence. [21] predicts a possible minimal lemma before dropping variables using the counterexample to propagation. These studies underscore the importance of effective lemma propagation in optimizing IC3 performance.

Proof obligation management, in turn, facilitates improved lemma propagation and contributes to the overall efficiency of IC3. [12] treats the negation of a discovered lemma as a may-proof-obligation to attempt possible lemma propagation and uses the reachable states that witness a failure of a may-proof-obligation to improve future lemma generalization. [20] presented a comprehensive study on proof obligation generalization techniques in IC3, including lifting, 01X-simulation, and justification-based generalization, highlighting the significant impact of different PO generalization methods on experimental performance.

Motivated by these insights, we introduce Critical Proof Obligations (CPOs), a novel concept aimed at improving the effectiveness of IC3 by prioritizing the handling of proof obligations that significantly influence the lemma propagation process. By identifying and focusing on these critical proof obligations, we streamline the verification process, boost lemma propagation, and improve the algorithm's convergence speed.

Contributions: This paper introduces the concept of CPO, which prioritizes highly impactful proof obligations to improve lemma propagation and frame refinement and presents two CPO-driven methods designed to enhance the IC3/PDR algorithm. The first method, CPO-Driven UNSAT Core Generation, optimizes UNSAT core generation by leveraging CPOs to prioritize relevant literals in SAT queries, effectively identifying CPOs that drive lemma propagation. The second method, CPO-Driven Proof Obligation Propagation, selectively propagates proof obligations based on their criticality, minimizing unnecessary lemma accumulation and promoting efficient verification.

We implemented our two novel CPO-driven methods in two well-performing IC3 safety model checkers: IC3ref [8] and modelchecker [3]. Additionally, we conducted experiments using HWMCC benchmarks to compare our solvers against three other state-of-the-art IC3/PDR algorithms. A comprehensive evaluation of the proposed methods demonstrates that identi-

*Corresponding author

fying CPOs boosts lemma propagation and proof efficiency, leading to substantial performance gains and validating the effectiveness of our approach.

Structure: Section II provides the necessary background information. Section III introduces the concept of CPO and two CPO-driven methods for optimizing proof obligation management. We evaluate the proposed methods in Section IV, discuss related work in Section V, and conclude the paper in Section VI.

II. PRELIMINARIES

This paper focuses on bit-level symbolic model-checking methods. The following subsections introduce fundamental concepts relevant to propositional logic, finite-state transition systems, safety checking, SAT solving, and the IC3 algorithm.

A. Propositional Logic

Given a set of Boolean variables $V = \{v_1, \dots, v_n\}$, a *literal* of a variable v is either the variable itself (v) or its negation ($\neg v$). We use $\text{var}(l)$ to denote the variable corresponding to a literal l .

A *clause* (resp. *cube*) is defined as the disjunction (resp. conjunction) of a set of literals. The negation of a cube forms a clause (also referred to as a *lemma*), and vice versa. If all literals of clause (resp. cube) d are included in clause (resp. cube) c , then d is called a sub-clause (resp. sub-cube) of c , denoted by $d \subseteq c$. The *conjunctive normal form* (CNF) represents a formula as a conjunction of clauses.

A full (resp. partial) *assignment* $\alpha : V \rightarrow \{\text{false}, \text{true}\}$ maps all (resp. some) variables to Boolean values and can be represented as a cube. We use $\alpha \models F$ (resp. $\alpha \not\models F$) to indicate that an assignment α satisfies (resp. falsifies) a formula F , making α a *model* of F . Additionally, $F \Rightarrow G$ implies that if F holds, G must also hold, meaning all models of F are also models of G .

B. Finite-state Transition System and Safety Checking

A *finite-state transition system* (FTS) is typically represented as a triple $S = \langle V, I, T \rangle$, where V is a set of Boolean variables, $I(V)$ is the initial state, and $T(V, V')$ is the transition relation describing the state transitions within the system. A state s of an FTS S is a cube composed of literals over V . We use the primed version s' to denote the next state of s according to $T(V, V')$. A state s is called an *F-state* if $s \models F$.

A *trace* of a finite-state transition system (FTS) S is a sequence of states $\gamma = s_0, s_1, s_2, \dots$, which can be either finite or infinite. The initial state s_0 must satisfy the initial condition I , and for each state pair s_i, s_{i+1} in the trace γ , the transition relation $s_i, s_{i+1}' \models T$ must hold. A state s is said to be *reachable* if it appears in any trace of the system S .

In formal verification, safety properties of transition systems are commonly verified through *safety model checking*. A safety property $P(V)$ asserts that only those states satisfying P (referred to as *P-states*) are reachable. The property P is considered an *invariant* for the system S if all reachable

states are *P-states*. If P is not invariant, there exists a finite counterexample trace $s_0, s_1, s_2, \dots, s_n$ with $s_0 \models \neg P$, where s_0 is referred to as a *bad state*.

A lemma c is called *inductive* if it satisfies the following two conditions:

- *Initiation:* c holds for all initial states, i.e., $I \Rightarrow c$.
- *Consecution:* c is closed under the transition relation, i.e., $c \wedge T \Rightarrow c$.

A lemma c is said to be *inductive relative* to F_i if $I \Rightarrow c$ and also ensures that $F_i \wedge c \wedge T \Rightarrow c$. This can be verified by checking whether the formula $F_i \wedge c \wedge T \wedge \neg c$ is unsatisfiable. An *inductive strengthening* of a safety property P is a formula F such that the conjunction $F \wedge P$ is inductive.

C. SAT Solving and Unsatisfiable Core

The propositional satisfiability (SAT) problem is a decision problem that determines whether a given propositional formula is satisfiable. SAT solvers are algorithms designed to solve this problem, typically taking inputs in *conjunctive normal form* (CNF).

Incremental SAT solvers are a specialized class of SAT solvers that can efficiently solve a sequence of related CNF formulas. One key feature of incremental SAT solvers is their ability to solve a formula while considering a set of *assumption* literals, which are frozen to true during the search process.

Given a CNF formula F and a set of assumptions A , if F is satisfiable, the solver returns “SAT” along with a satisfying assignment. If F is unsatisfiable, the solver returns “UNSAT” and a subset A_o of A , known as the UNSAT core, representing the reason for the formula’s unsatisfiability.

For clarity in describing the following algorithms, we assume that the SAT solver used in this paper provides the following APIs for incremental SAT solving [16]:

- **is_SAT(F, A):** Checks the satisfiability of a propositional formula F under the assumption A . This can be expressed as $\text{is_SAT}(\overline{F})$, where $\overline{F} = F \wedge A$.
- **get_model():** If $\text{is_SAT}(F, A)$ returns “SAT”, this API returns a model for F under the assumption A . We use $\text{val}(v)$ to denote the value of the variable v .
- **get_core():** If $\text{is_SAT}(F, A)$ returns “UNSAT”, this function returns the UNSAT core A_o for F under the assumption A .

D. IC3 Algorithm Overview

The IC3 algorithm, also known as PDR, is an efficient model-checking technique designed to verify safety properties in finite-state transition systems. A high-level overview of IC3/PDR is presented in Alg. 1.

IC3 incrementally constructs an inductive proof of correctness using a sequence of frames, denoted as $F_0, F_1, F_2, \dots$, where each frame F_i over-approximates the states reachable within i steps from the initial state. The algorithm starts with $F_0 = I$, representing the initial state set. It builds subsequent frames such that $F_i \Rightarrow F_{i+1}$ and $F_i \wedge T \Rightarrow F_{i+1}$, progressively strengthening them to ensure no bad states are reachable. The goal is to eventually obtain a frame F that is safe and

inductive, i.e., it contains no bad states and is closed under the transition relation, thereby proving the system's safety.

IC3 iteratively selects a cube s in F such that it transitions to a bad state in one step. Such an s is identified as a counterexample to induction (CTI), which starts from F and transitions to a bad state. IC3 attempts to block CTIs by maintaining a proof obligation (PO) queue and adding new lemmas to the frames. Each PO is a pair (s, i) , where s is a CTI and i is a frame number, indicating that s must be proven inductively relative to F_i ; otherwise, the property fails. If s is inductively relative to F_i , meaning it is unreachable within $i + 1$ steps from the initial state, we say (s, i) is blocked.

The blocking process may involve the recursive generation of proof obligations, which are propagated backward through frames until reaching a safe resolution or the initial state. Once blocking ensures that F cannot reach a bad state, the propagation phase begins, pushing a lemma c in $F_i \setminus F_{i-1}$ to F_{i-1} if it is inductive relative to F_{i-1} . When two consecutive frames become identical ($F_i = F_{i-1}$), an inductive invariant is established, and the model's safety is proved.

III. LEVERAGING CRITICAL PROOF OBLIGATIONS

In the IC3 algorithm, managing and effectively processing proof obligations (POs) is crucial for ensuring the system's safety, as it enables iterative refinement of frames to exclude bad states. In this section, we present our methodology for enhancing the IC3/PDR algorithm by evaluating and prioritizing proof obligations to promote effective lemma propagation during frame refinement. This will thereby accelerate convergence towards proving system safety.

A. Intuition behind Critical Proof Obligations

As discussed in the preliminaries, a proof obligation (PO) is defined as a pair (s, i) , where s represents a counterexample to induction (CTI) leading to a bad state and should be proved inductive relative to F_i . In the IC3/PDR framework, blocking all possible states that may lead to a bad state within a frame is impractical due to the prohibitive computational cost. Instead, the primary objective is to ensure that each blocked counterexample contributes a useful lemma that facilitates lemma propagation and the construction of system invariants. Specifically, for a lemma $\neg c$ in $F_i \setminus F_{i-1}$, and a CTI s such that $c \subseteq s$, the successful processing of the PO (s, i) —i.e., showing that s is inductive relative to frame F_i and adding the lemma $\neg s$ to frame F_i —is crucial for propagating the lemma $\neg c$ from frame F_i to frame F_{i-1} . This motivates the formal definition of a critical proof obligation (CPO).

Definition 1 (Critical Proof Obligation). *Let (s, i) be a proof obligation, where s represents a counterexample to induction (CTI) leading to a bad state, and i is a frame index. We define (s, i) as a Critical Proof Obligation if at least one lemma $\neg c$ exists in $F_i \setminus F_{i-1}$ such that $c \subseteq s$.*

Theorem 1. *Let $\neg c$ be a lemma in $F_i \setminus F_{i-1}$. The lemma $\neg c$ can be propagated to F_{i-1} if only all critical proof obligations (s, i) such that $c \subseteq s$ are blocked.*

Algorithm 1: Overview of IC3 and our methods

```

/* Codes related to CPO-Driven UNSAT Core
   Generation in Sect. III-B (resp. CPO-Driven
   Proof Obligation Propagation in Sect. III-C)
   is colored red (resp. blue). */
1 function inductive_relative(cube c, level l):
2   return not is_SAT( $F_l \wedge \neg c \wedge T, c'$ );
3
4 function generalize(cube c, level l):
5   for each  $p \in c$  do
6     if  $I \Rightarrow \neg c \setminus \{p\}$  and inductive_relative( $c \setminus \{p\}, l$ ) then
7        $c := c \setminus \{p\}$ ;
8   return c;
9
10 function get_predecessor(cube s, level l):
11   input,  $t \leftarrow \{i \in \text{get\_model}() \mid \text{var}(i) \in V\}$ ;
12   assert(not is_SAT(input  $\wedge T \wedge \neg s'$ , reverse_sort( $t$ )));
13   pre := get_core();
14   for each  $\neg c \in F_{l-1} \setminus F_l$  do
15     if  $c \subseteq \text{pre}$  then
16       reward_core(pre);
17     Break;
18   return pre;
19
20 function block_obligation():
21   while not obs.empty() do
22      $\langle c, l \rangle \leftarrow \text{obs.pop}()$ ;
23     if inductive_relative( $c, l$ ) then
24       if  $I \wedge \text{get\_core}()$  then  $s := \text{generalize}(c, l)$ ;
25       else  $s := \text{generalize}(\text{get\_core}(), l)$ ;
26       while  $l \leq k$  and inductive_relative( $s, l$ ) do
27          $l := l + 1$ ;
28       for each  $\neg t \in F_l \setminus F_{l+1}$  do
29         if  $t \subseteq c$  and  $l \leq k$  then
30           obs.insert( $\langle c, l \rangle$ );
31         Break;
32       for  $i \in [1, l]$  do  $F_i := F_l \cup \{\neg s\}$ ;
33       else if  $l = 0$  then return false;
34       else obs.insert( $\langle \text{get\_predecessor}(c, l), l - 1 \rangle$ );
35   return true;
36
37 function propagate(level l):
38   for  $i \in [1, l]$  do
39     for  $\neg c \in F_i \setminus F_{i+1}$  do
40       if inductive_relative( $c, i$ ) then
41          $F_{i+1} := F_{i+1} \cup \{\neg c\}$ ;
42       if  $F_i = F_{i+1}$  then return true;
43   return false;
44
45 procedure ic3( $I, T, P$ ):
46   if is_SAT( $I \wedge \neg P$ ) then return unsafe;
47    $F_0 \leftarrow I, k \leftarrow 1, F_k \leftarrow T$ ;
48   while true do
49     while is_SAT( $F_k \wedge \neg P$ ) do
50        $c := \text{get\_predecessor}(\text{get\_model}(), k)$ ;
51       obs.insert( $\langle c, k - 1 \rangle$ );
52       if not block_obligation() then return "unsafe";
53        $k + 1, F_k \leftarrow T$ ;
54   if propagate( $k$ ) then return "safe";

```

Proof. Assume a proof obligation (s, i) exists such that $c \subseteq s$ is not blocked. Then, there exists a state t in F_i such that $t \wedge T \Rightarrow s'$. Since $\neg c$ is present in F_i , we have $t \wedge \neg c \wedge T \Rightarrow t \wedge T \Rightarrow s' \Rightarrow c'$. Therefore, there exists a state t in F_i such that $t \wedge \neg c \wedge T \Rightarrow c'$, implying that the lemma $\neg c$ is not inductive relative to F_i , and thus cannot be propagated to F_i .

This contradicts the original claim that $\neg c$ can be propagated to F_i . Therefore, our assumption is false, and all critical proof obligations (s, i) such that $c \subseteq s$ must be blocked for $\neg c$ to propagate. $\square$

Critical Proof Obligations are crucial because they directly impact the propagation of lemmas across frames, which is fundamental to the success of the IC3 algorithm. Our approach aims to identify and leverage CPOs and prioritize their impact to effectively propagate lemmas, refine frames, and contribute to verification. The following subsections introduce two methods for leveraging and managing Critical Proof Obligations for optimal impact.

B. CPO-Driven UNSAT Core Generation

The branching strategy is a crucial feature of modern Conflict-Driven Clause Learning (CDCL) [16] SAT solvers. Traditional branching heuristics, like Variable State Independent Decaying Sum (VSIDS) [17], play an important role in prioritizing decision variables based on their involvement in conflicts, ensuring that frequently involved variables are selected to drive the solver towards resolution.

In our approach, we adopt a similar idea to influence the generation of UNSAT cores by ordering the assumption literals in SAT queries according to their scores (sort from high to small, see line 12 of Alg. 1). This score reflects how often each literal appears in recent Critical Proof Obligations. The key observation is that modern SAT solvers apply the assumption literals in the order provided by the user. Consequently, the UNSAT core produced when the formula is unsatisfiable depends heavily on this order, with literals that appear earlier being more likely to be included in the core.

Our goal is to ensure that each returned UNSAT core leads to the generation of a Critical Proof Obligation. To achieve this, we keep track of the UNSAT cores that successfully produce CPOs and heuristically adjust the input assumption order, aligning the variables in the assumption list with those frequently observed in successful CPO-generating cores. To facilitate this process, we introduce a metric called *Assumption Core Consistency (ACC)* to evaluate the consistency of variable occurrence in CPOs.

Definition 2 (Assumption Core Consistency). *Let A be the set of assumptions, and let $O(a)$ denote the index of $\text{var}(a)$ in the default assumption ordering. Let $C = \{a_1, \dots, a_n\} \subset A$ represent the UNSAT core obtained during the CTI search, and let $i \in \{1, \dots, n\}$ denote the index of variable $\text{var}(a_i)$ in C . The rules governing UNSAT core generation hold that $i \leq O(\text{var}(a_i))$ for all variables in the core.*

The ACC score for a variable $v \in \{\text{var}(l) \mid l \in C\}$ is defined as the difference between its index in the UNSAT core and its original position in the assumption list:

$$\text{acc}(\text{var}(a_i)) = O(\text{var}(a_i)) - i$$

A higher ACC score indicates greater inconsistency in the ordering, suggesting a need for adjustment.

ACC serves as a basis for refining the assumption ordering to improve UNSAT core generation. The refinement process is outlined below (function *get_predecessor* of Alg. 1):

ACC-based Generalization Refinement: After identifying a predecessor state t of the CTI s in frame F_l , the SAT solver produces an UNSAT core pre , which serves as a generalized version of t and forms a new proof obligation $(pre, l - 1)$. For each variable $v \in \{\text{var}(l) \mid l \in pre\}$, we calculate its ACC score $\text{acc}(v)$ and adjust its sorting score $sc(v)$ only if $(pre, l - 1)$ is identified as a Critical Proof Obligation. This ensures that variable scores are updated exclusively when the associated proof obligation is crucial for lemma propagation across frames.

The score is updated as follows (line 16 of Alg. 1):

$$sc(v) := sc(v) + \text{acc}(v)$$

This adjustment improves UNSAT core generation by compacting the trail of assumptions to effectively refute the given assumptions and leveraging CPO historical data to identify better UNSAT cores and enhance lemma propagation.

C. CPO-Driven Proof Obligation Propagation

In the IC3/PDR algorithm, proof obligation propagation is critical in frame refinement by generating lemmas that block bad states. Traditionally, all proof obligations are propagated to the latest frame, which increases the chance of finding counterexample paths further from the current frame. However, several studies ([5, 12]) have indicated that unguided lemma learning can accumulate redundant lemmas. Similarly, indiscriminate proof obligation propagation can result in excessive lemmas generated from blocking distant counterexamples, which are unreachable in the current frame, making it more challenging to propagate all lemmas within the same frame to prove system safety.

Therefore, our approach leverages Critical Proof Obligations for a more efficient trade-off. Instead of propagating all proof obligations, we selectively push forward CPOs that are directly linked to learned lemmas in the corresponding frames. This targeted approach ensures that only the most impactful proof obligations contribute to lemma propagation, reducing the production of unnecessary lemmas while still facilitating the discovery of critical counterexample paths.

The implementation of CPO-Driven Proof Obligation Propagation (function *block_obligation* in Alg. 1) is straightforward, as it builds upon the existing IC3 framework with minimal changes to the core algorithm. When a proof obligation (s, i) is blocked, we first check for any lemma $\neg c$ in frame F_i such that $c \subseteq s$. If such a lemma exists, we identify (s, i) as

TABLE I: Tested tools and configurations in the experiments.

Tools	IC3ref	MCer	ABC	CAV23	DAC24
Available	-uc	-uc			
Flags	-pr	-pr	-pdr	-	-

a CPO, generalize s to F_l , and push the CPO to frame F_l (see line 23–31 of Alg. 1). If no such lemma is found, the PO is generalized and discarded. By selectively propagating CPOs, our method enhances lemma propagation efficiency and accelerates convergence, as shown in Section IV-C.

IV. EVALUATION

A. Experimental Setup

We implemented our two proposed methods—CPO-Driven UNSAT Core Generation and CPO-Driven Proof Obligation Propagation—on two top-performing baseline solvers, using the flags “-uc” and “-pr” to enable each method, respectively:

- **IC3ref** [8]. Developed by the original inventor of IC3, this solver is competitive and commonly used as a baseline in recent studies [19, 21, 22].
- **modelchecker** [3]. This tool implements IC3 using the advanced incremental SAT solver CaDiCaL [7] and is referred to as MCer for brevity.

Additionally, we selected three state-of-the-art open-source IC3/PDR algorithms for comparison:

- **ABC** [1]. This tool includes the well-known implementation of the PDR algorithm.
- **IC3ref-CAV23** [22]. This version enhances IC3Ref by controlling lemma generalization to discover i -good lemmas and is referred to as “CAV23” for brevity.
- **IC3ref-DAC24** [21]. This version applies a generalization technique based on lemma prediction within IC3ref and is referred to as “DAC24” for brevity.

Table I lists the available configurations for the tools mentioned above.

We conducted our experiments using the AIGER format benchmarks from the Hardware Model Checking Competitions (HWMCC) of 2019, 2020, and 2024, totaling 786 instances. All benchmarks were converted to AIGER version 1.8 [6] using the “aigmove” command from the AIGER tool suite [2] to ensure compatibility with open-source model-checking tools, such as ABC, which have limitations in handling invariant constraints.

Since all solvers discussed in this paper are deterministic, each instance was run once on a dedicated CPU core, with a wall time limit of 5000 seconds and an 8 GB memory limit. Experiments were conducted on a cluster with two AMD EPYC 7763 CPUs (128 cores, 1024 GB RAM) running CentOS 7.7.1908. All solvers were implemented in C/C++ and compiled with GNU gcc/g++ 9.4.0. To ensure reproducibility, the source code for our optimized solvers is available at <https://github.com/IC3Contributor/DAC25>.

TABLE II: Performance Summary among different configurations. Versions with our proposed optimizations are marked with “-uc”, “-pr”, or both.

name	#solved	#safe	#unsafe	#gain	#lost
IC3ref	463	388	75	-	-
IC3ref -uc	478	397	81	19(11/8)	4(2/2)
IC3ref -pr	471	392	79	11(6/5)	3(2/1)
IC3ref -uc -pr	481	397	84	20(11/9)	2(2/0)
MCer	455	380	75	-	-
MCer -uc	461	388	73	18(14/4)	12(6/6)
MCer -pr	461	383	78	11(5/6)	5(2/3)
MCer -uc -pr	470	394	76	25(19/6)	10(5/5)
ABC -pdr	465	398	67	-	-
CAV23	476	392	84	-	-
DAC24	466	390	76	-	-

B. Experimental Results

The experimental results are summarized in Table II, Fig. 1, and Fig. 2, which present the overall performance, configuration comparisons, and runtime scatter plots of implementations with and without our proposed optimizations, respectively. By incorporating our optimizations, different implementations solved more cases than the original versions across various time limits, achieving higher efficiency in solving a more significant number of instances. Notably, IC3ref-uc-pr and MCer-uc-pr solved 18 and 15 additional instances, respectively, demonstrating significant gains.

Compared to three state-of-the-art IC3/PDR algorithms, our best configuration, IC3ref-uc-pr, solved 16, 5, and 15 more instances than ABC, CAV23, and DAC24, respectively. Specifically, for unsafe instances, IC3ref-uc performed on par with CAV23, solving most cases, while for safe instances, IC3ref-uc-pr achieved similar performance to the best competitors (ABC), with only one instance difference. IC3ref-uc-pr showed notable improvements in verifying both safe and unsafe instances, achieving better results with fewer losses compared to the baseline and competing solvers.

In conclusion, the performance summary table, Cumulative Distribution Function (CDF), and scatter plots consistently demonstrated runtime improvements with our methods. Both IC3ref-uc-pr and MCer-uc-pr achieved significant speedups across multiple instances, underscoring the effectiveness of our optimizations in enhancing IC3’s performance and efficiency.

C. Further Analysis

To demonstrate the effectiveness of our methods, we computed two metrics for each instance: the CPO Identification Rate ($SR_o = N_o/N_{ti}$) and the Lemma Propagation Success Rate ($SR_l = N_l/N$). Here, N_o is the number of CTIs identified as CPOs, N_{ti} is the total number of generated CTIs, N_l is the number of successful lemma propagation attempts, and N is the total number of lemma propagation attempts. The average SR_o for MCer, MCer-

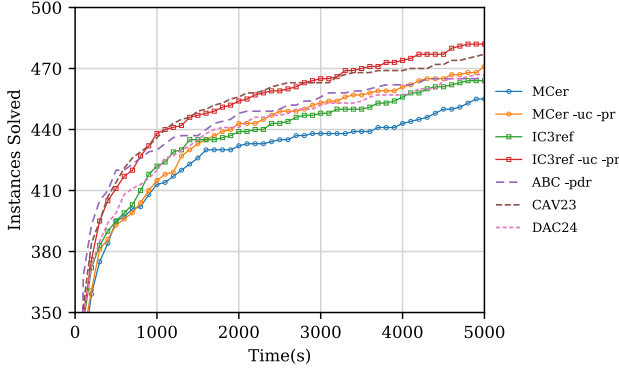


Fig. 1: Cumulative distribution function (CDF) comparing the performance of our improved solvers with state-of-the-art solvers (for clarity, results for single heuristic configurations are omitted).

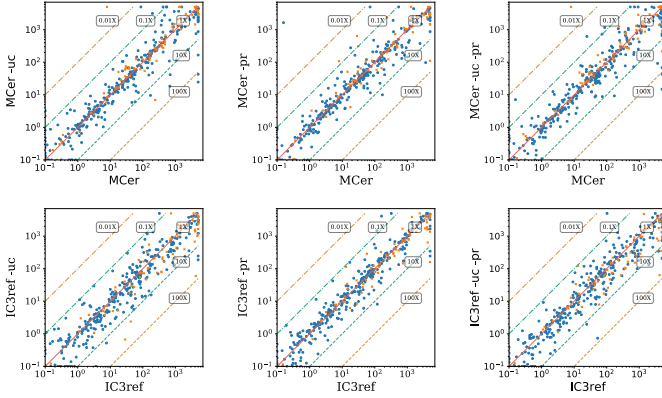


Fig. 2: Scatter plots comparing runtimes of MCer/IC3ref and their optimized versions. Points below the diagonal indicate better performance, with orange for unsafe instances and blue for safe instances.

uc-pr, IC3ref, and IC3ref-uc-pr were 0.4962, 0.5378, 0.2657, and 0.3152, respectively, indicating that our optimizations consistently improved CPO identification. Likewise, the average SR for these solvers were 0.7052, 0.7259, 0.6585, and 0.7056, showing enhanced lemma propagation efficiency and accelerated convergence.

Fig. 3 visualizes the impact of our methods through a scatter plot, comparing the deviation in SR (y-axis) with the deviation in verification time (x-axis) for MCer/IC3ref with and without our optimizations. For each instance, let t and t' represent the verification times for MCer/IC3ref and MCer-uc-pr/IC3ref-uc-pr, respectively, and sr and sr' denote their corresponding CPO Identification Rates. Each point on the plot is defined by $t - t'$ on the x-axis and $sr' - sr$ on the y-axis. More points in the first quadrant provide more substantial evidence for the effectiveness of our method, as they indicate higher CPO identification rates and shorter verification times. The results in Fig. 3 for both MCer and IC3ref clearly demonstrate that identifying CPOs contributes to improved model-checking performance by reducing verification time.

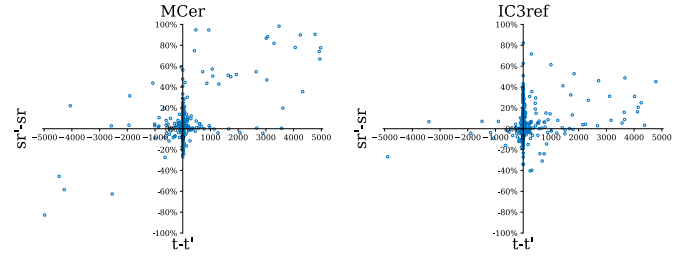


Fig. 3: Scatter plot illustrating the impact of our optimizations on verification time and CPO Identification Rate.

V. RELATED WORKS

SAT-based model-checking algorithms are fundamental in hardware verification, driving the development of various IC3 [8] and PDR [10] variants to improve efficiency and applicability. For example, [18] proposed a forward/backward hybrid version of PDR, and [4] introduced IC3 integrated with a specialized SAT solver called “SAT Modulo SAT”. Additionally, kAvy [14] combined IC3 with k -induction to enhance its capabilities.

Lemma propagation involves propagating learned lemmas through different frames to block invalid states and accelerate convergence towards proving safety. [13] introduced counterexamples to generalization to enhance blocking bad states and lemma propagation; [21] refined failed pushed parent lemmas as predictions to avoid dropping variables. Our approach differs from these works by introducing Critical Proof Obligations (CPOs) to optimize proof obligation management, thereby enhancing lemma propagation and frame refinement.

Another related work is QUIP [12], which treats the negation of a discovered lemma c as a may-proof-obligation, attempting to push it to the next frame to propagate more lemmas. While both QUIP and our method focus on proof obligation management and selective lemma propagation, our approach evaluates and prioritizes existing proof obligations that directly impact lemma propagation across frames rather than introducing new ones.

VI. CONCLUSIONS AND FUTURE WORKS

This work enhances the IC3 algorithm’s efficiency through Critical Proof Obligations and two CPO-driven methods that optimize proof obligation handling and lemma propagation. Extensive experiments demonstrated that our methods significantly improved the identification of impactful proof obligations and increased the success rate of lemma propagation, highlighting their effectiveness in enhancing overall performance. In the future, we plan to design IC3 algorithms that tightly cooperate with SAT solvers, extend our techniques to word-level IC3 tools, and adapt CPO-driven methods for broader application in other model-checking algorithms.

ACKNOWLEDGMENT

This work was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDA0320000 and XDA0320300.

REFERENCES

- [1] ABC. <https://github.com/berkeley-abc/abc>
- [2] AIGER Tools. <https://fmv.jku.at/aiger/aiger-1.9.9.tar.gz>
- [3] Modelchecker. <https://github.com/ISCAS-modelchecker/modelchecker>
- [4] Bayless, S., Val, C.G., Ball, T., Hoos, H.H., Hu, A.J.: Efficient modular SAT solving for IC3. In: FMCAD. pp. 149–156. IEEE (2013)
- [5] Berryhill, R., Veira, N., Veneris, A., Poulos, Z.: Learning lemma support graphs in quip and ic3. In: 2017 IEEE 2nd International Verification and Security Workshop (IVSW). pp. 105–110 (2017). <https://doi.org/10.1109/IVSW.2017.8031554>
- [6] Biere, A.: The AIGER And-Inverter Graph (AIG) format version 20071012. Tech. Rep. 07/1, Institute for Formal Models and Verification, Johannes Kepler University, Altenbergerstr. 69, 4040 Linz, Austria (2007)
- [7] Biere, A., Fazekas, K., Fleury, M., Heisinger, M.: CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. In: Balyo, T., Froleys, N., Heule, M., Iser, M., Järvisalo, M., Suda, M. (eds.) Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions. Department of Computer Science Report Series B, vol. B-2020-1, pp. 51–53. University of Helsinki (2020)
- [8] Bradley, A.R.: Sat-based model checking without unrolling. In: VMCAI. Lecture Notes in Computer Science, vol. 6538, pp. 70–87. Springer (2011)
- [9] Clarke, E.M., Henzinger, T.A., Veith, H., Bloem, R. (eds.): Handbook of Model Checking. Springer (2018)
- [10] Eén, N., Mishchenko, A., Brayton, R.K.: Efficient implementation of property directed reachability. In: FMCAD. pp. 125–134. FMCAD Inc. (2011)
- [11] Griggio, A., Roveri, M.: Comparing different variants of the ic3 algorithm for hardware model checking. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. **35**(6), 1026–1039 (2016)
- [12] Gurfinkel, A., Ivrii, A.: Pushing to the top. In: FMCAD. pp. 65–72. IEEE (2015)
- [13] Hassan, Z., Bradley, A.R., Somenzi, F.: Better generalization in IC3. In: FMCAD. pp. 157–164. IEEE (2013)
- [14] Krishnan, H.G.V., Vizel, Y., Ganesh, V., Gurfinkel, A.: Interpolating strong induction. In: CAV (2). Lecture Notes in Computer Science, vol. 11562, pp. 367–385. Springer (2019)
- [15] Li, J., Dureja, R., Pu, G., Rozier, K.Y., Vardi, M.Y.: Simplecar: An efficient bug-finding tool based on approximate reachability. pp. 37–44. Springer (2018)
- [16] Marques-Silva, J., Lynce, I., Malik, S.: Conflict-driven clause learning SAT solvers. In: Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 336, pp. 133–182. IOS Press (2021)
- [17] Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S.: Chaff: Engineering an efficient sat solver. In: Proceedings of the 38th annual Design Automation Conference. pp. 530–535 (2001)
- [18] Seufert, T., Scholl, C.: Combining PDR and reverse PDR for hardware model checking. In: DATE. pp. 49–54. IEEE (2018)
- [19] Seufert, T., Scholl, C., Chandrasekharan, A., Reimer, S., Welp, T.: Making PROGRESS in property directed reachability. In: MBMV. pp. 1–2. VDE/IEEE (2022)
- [20] Seufert, T., Winterer, F., Scholl, C., Scheibler, K., Paxian, T., Becker, B.: Everything you always wanted to know about generalization of proof obligations in pdr. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **42**(4), 1351–1364 (2023). <https://doi.org/10.1109/TCAD.2022.3198260>
- [21] Su, Y., Yang, Q., Ci, Y.: Predicting lemmas in generalization of ic3. In: Proceedings of the 61st ACM/IEEE Design Automation Conference. pp. 1–6 (2024)
- [22] Xia, Y., Becchi, A., Cimatti, A., Griggio, A., Li, J., Pu, G.: Searching for i-good lemmas to accelerate safety model checking. In: CAV (2). Lecture Notes in Computer Science, vol. 13965, pp. 288–308. Springer (2023)